



Міністерство освіти і науки України Національний технічний
університет України

“Київський політехнічний інститут імені Ігоря Сікорського” Факультет
інформатики та обчислювальної техніки Кафедра інформаційних систем та
технологій

ЛАБОРАТОРНА РОБОТА №8

з дисципліни "Технології розроблення програмного забезпечення"

Тема: «Веб-сервіс автоматизованої перевірки програм лабораторного
практикуму на мові програмування Java »

Виконав

студент групи ІА–33:

Заранік М.Ю

Перевірив:

Мягкий М.Ю.

Зміст

Вступ	3
Мета роботи	3
Завдання роботи	3
Теоретичні відомості	4
Хід роботи	7
Питання до лабораторної роботи	21
Висновки	23

Вступ

Мета роботи

Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

Завдання

1. Ознайомитись з короткими теоретичними відомостями.
2. Реалізувати частину функціоналу робочої програми у вигляді класів та
3. їхньої взаємодії для досягнення конкретних функціональних можливостей.
4. Реалізувати один з розглянутих шаблонів за обраною темою.
5. Реалізувати не менше 3-х класів відповідно до обраної теми.
6. Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт
7. повинен містити: діаграму класів, яка представляє використання шаблону в
8. реалізації системи, навести фрагменти коду по реалізації цього шаблону

Теоретичні відомості

Будь-який патерн проєктування, що застосовується під час створення програмних систем, являє собою формалізований опис часто повторюваних задач, ефективне рішення для цих задач і рекомендації щодо його використання у різних ситуаціях. Особливо це стосується таких структурних і поведінкових шаблонів, як Composite, Flyweight, Interpreter та Visitor. Кожен із них має усталене найменування і дозволяє багаторазово застосовувати перевірені архітектурні підходи. Важливим початковим етапом роботи з цими патернами є коректне моделювання предметної області, що забезпечує правильно сформульовану задачу й обґрунтований вибір шаблону.

Використання таких патернів дає низку переваг. Модель системи, створена з урахуванням Composite чи Flyweight, виділяє ключові структури та зв'язки, тоді як Interpreter і особливо Visitor роблять поведінку системи прозорою та зручною для розширення. Така модель залишається наочною, але водночас дозволяє гнучко проєктувати архітектуру, використовуючи формальний опис операцій і структур елементів. Патерни істотно підвищують адаптивність рішення, спрощують модифікацію і підтримку.

Особливу роль вони відіграють при інтеграції підсистем організації. Сукупність таких шаблонів утворює спільний словник проєктування для розробників. Зокрема, патерн Visitor виступає перевіреним підходом для відокремлення операцій від структури даних, що дозволяє додавати нову функціональність без зміни самих елементів. Таким чином, ці патерни є практичними архітектурними «ескізами», які забезпечують ефективність та узгодженість у відповідних ситуаціях.

8.2.1. Шаблон «Visitor»

Призначення: дозволяє додавати нові операції до складних структур даних (наприклад, Composite), не змінюючи класи їх елементів. Це забезпечує чітке розділення структури та поведінки.

Проблема: коли система містить багато різних елементів, виникає потреба регулярно додавати нову логіку обробки. У класичній структурі це призводить до постійних змін у великій кількості класів.

Рішення: створюється окремий об'єкт-відвідувач, який інкапсулює операції. Елементи структури (у тому числі Flyweight-елементи або вузли Interpreter) приймають відвідувача й передають йому управління.

Переваги: полегшує розширення поведінки, зменшує модифікації існуючих класів, покращує підтримуваність системи.

Недоліки: складніше додавати нові типи елементів структури, оскільки всі відвідувачі мають бути оновлені.

8.2.2. Шаблон «Interpreter»

Призначення: патерн «Interpreter» визначає спосіб опису граматики та механізм обробки виразів, дозволяючи змінювати її незалежно. Проблема: реалізація власної мови для конфігурацій або правил може призвести до величезної кількості умовних конструкцій і складної підтримки. Рішення: створюються класи-вирази двох ієрархій — термінальних і нетермінальних, а дерево Composite-структури виконується через метод interpret(). Ці вузли можуть взаємодіяти з Visitor для додаткових операцій. Переваги: чітке розділення граматики й логіки виконання.

8.2.3. Шаблон «Composite»

Призначення: дозволяє об'єднувати об'єкти в ієрархічну деревоподібну структуру та працювати з окремими елементами і цілими групами однаково.

Проблема: коли система містить складні вкладені структури (меню, файли, елементи інтерфейсу), виникає потреба уніфікувати роботу з ними, не розділяючи код на десятки

спеціальних перевірок.

Рішення: вводяться два типи об'єктів — Leaf і Composite, які мають спільний інтерфейс. Composite містить колекцію дочірніх елементів і делегує їм виконання операцій.

Переваги: спрощує обробку дерев, дозволяє рекурсивні структури, добре поєднується з Visitor.

Недоліки: складніше контролювати дозволені типи елементів, можливе надмірне узагальнення..

8.2.4. Шаблон «Interpreter»

Призначення: патерн «Interpreter» дозволяє винести опис граматики та алгоритм її обробки в окрему ієрархію класів, залишаючи конкретну інтерпретацію різних елементів структури підкласам.

Проблема: при додаванні нових правил або конструкцій у власну мову запитів чи конфігурацій код ускладнюється через численні умовні розгалуження та повторювану логіку.

Рішення: загальні кроки інтерпретації описуються у базових виразах, а специфічні частини реалізуються у підкласах термінальних і нетермінальних елементів, що формують Composite-дерево.

Переваги: граматика описана чітко й структуровано. Легко додавати нові правила та комбінувати операції зі схемами Visitor.

Недоліки: може зростати кількість класів при великій кількості елементів граматики.

Хід роботи

Патерн Visitor дозволяє додавати нові операції до складної Composite-структури (наприклад, етапів конвеєра), не модифікуючи їхні класи. Це забезпечує централізовану логіку обробки, розширюваність та спрощує масштабування системи.

У нашій системі підсистема складається з кількох елементів, які приймають відвідувача:

- BuildStageExecutor — етап збірки;
- TestStageExecutor — етап тестування;
- LinterStageExecutor — етап аналізу коду.

Роль відвідувачів виконують окремі класи:

- JsonReportVisitor — генерує JSON-звіт;
- XMLReportVisitor — формує XML-звіт;
- Visitor — спільний інтерфейс для всіх видів відвідувачів.

Зовнішній код працює лише через виклик `executor.accept(visitor)`, після чого відвідувач сам формує звіт відповідного формату. Завдяки цьому нові формати звітів можна додавати без змін у виконавцях етапів.

Переваги:

відокремлення логіки формування звітів від логіки виконання етапів;

спрощення розширення системи новими операціями;

можливість додавати підтримку нових форматів звітів без змін у класах етапів.

Недоліки:

при додаванні нових типів етапів потрібно змінювати всіх існуючих відвідувачів;

може виникнути надмірна кількість відвідувачів при великій кількості форматів.

2. Реалізація класів

Для реалізації патерну Visitor створено набір класів, які інкапсулюють логіку обробки результатів роботи етапів.

JsonReportVisitor формує JSON-звіт, перетворюючи логи кожного етапу у валідні JSON-структури.

XMLReportVisitor генерує XML-звіт, забезпечуючи екранування спецсимволів і зручне представлення даних.

Обидва класи реалізують інтерфейс Visitor, який визначає методи visitBuild, visitTest, visitLint і generateReport().

Класи етапів (BuildStageExecutor, TestStageExecutor, LinterStageExecutor) викликають відповідний метод відвідувача, передаючи свої логи.

Завдяки цьому клієнтський код працює лише з відвідувачем, а не з внутрішньою логікою етапів, що робить систему гнучкою, розширюваною і простою в супроводі.

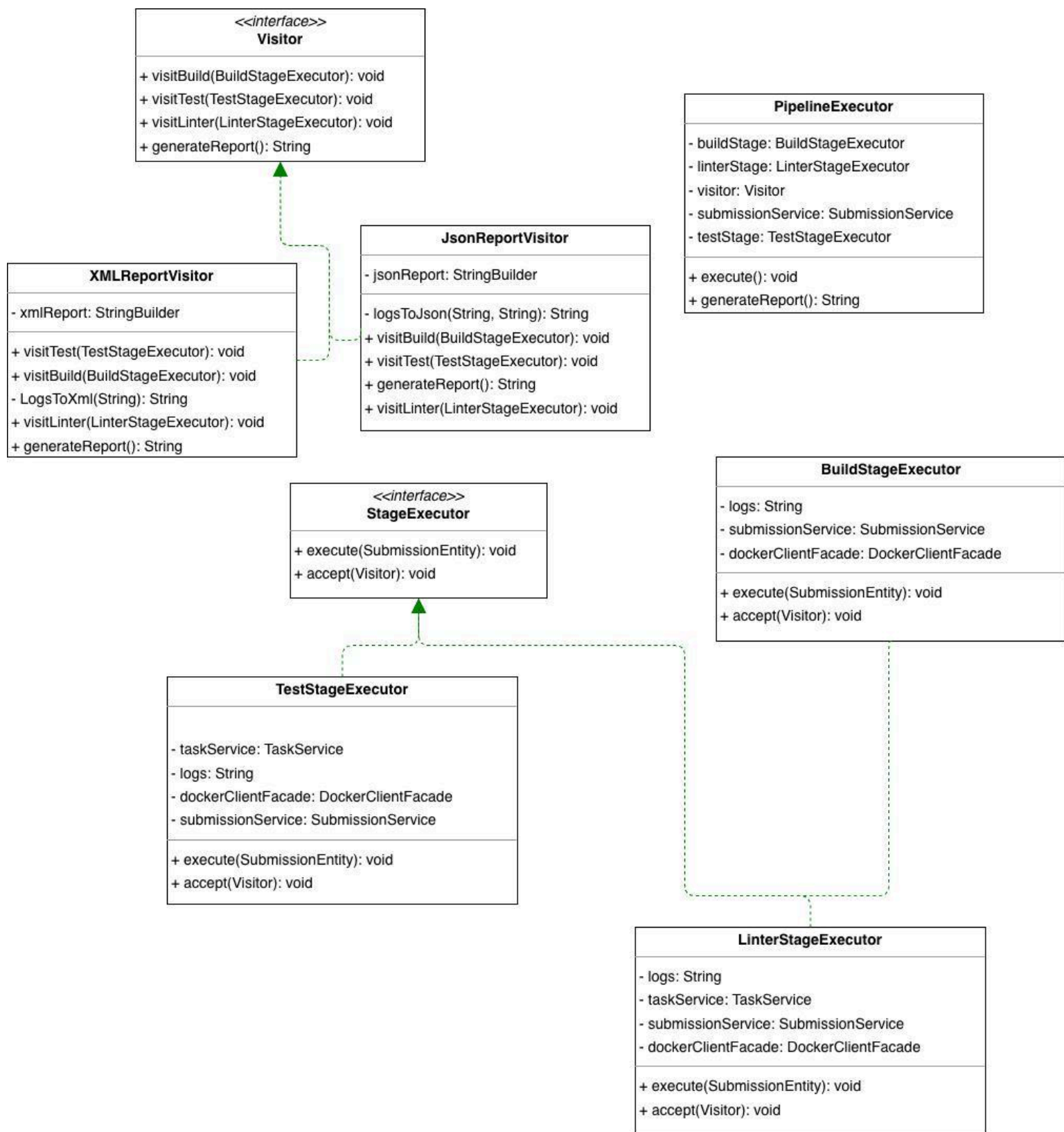
3. Послідовність виконання

Послідовність роботи системи з патерном Visitor виглядає наступним чином. Зовнішній код, тобто клієнт, взаємодіє виключно з єдиною точкою — викликом відвідувача для формування звіту, наприклад, visitor.generateReport(), і не турбується про внутрішні деталі роботи етапів конвеєра: виконання build, тестування чи лінтингу.

Клієнт лише передає відвідувача об'єктам етапів, а вся складна логіка обробки даних прихована всередині реалізацій Visitor.

Visitor виступає координатором обробки результатів між різними компонентами системи. Спочатку кожен етап конвеєра (BuildStageExecutor, TestStageExecutor, LinterStageExecutor) приймає відвідувача та передає йому свої логи.

Після цього конкретна реалізація Visitor (наприклад, JsonReportVisitor або XMLReportVisitor) обробляє ці дані, формує потрібні структури та додає їх до загального звіту. Після проходження всіх етапів Visitor генерує фінальний результат у відповідному форматі.



4. Діаграма класів

рис 1.1 Діаграма класів системи

5. Код системи який реалізує паттерн Visitor

Visitor

```
package com.example.demo.service.executor.visitor;

import com.example.demo.service.executor.stage.BuildStageExecutor;
import com.example.demo.service.executor.stage.LinterStageExecutor;
import com.example.demo.service.executor.stage.TestStageExecutor;

public interface Visitor {
    void visitBuild(BuildStageExecutor build);
    void visitTest(TestStageExecutor test);
    void visitLinter(LinterStageExecutor linter);
    String generateReport();
}
```

JsonReportVisitor

```
package com.example.demo.service.executor.visitor;

import com.example.demo.service.executor.stage.BuildStageExecutor;
import com.example.demo.service.executor.stage.LinterStageExecutor;
import com.example.demo.service.executor.stage.TestStageExecutor;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Component;

@Primary
@Component("JsonVisitor")
public class JsonReportVisitor implements Visitor {
    private final StringBuilder jsonReport = new StringBuilder();

    @Override
    public void visitBuild(BuildStageExecutor build) {
        String logs = build.getLogs();
        jsonReport.append(logsToJson("build", logs));
    }

    @Override
    public void visitTest(TestStageExecutor test) {
        String logs = test.getLogs();
        jsonReport.append(logsToJson("test", logs));
    }

    @Override
    public void visitLinter(LinterStageExecutor linter) {
        String logs = linter.getLogs();
        jsonReport.append(logsToJson("linter", logs));
    }
}
```

```

@Override
public String generateReport() {
    return getJsonReport();
}

private String logsToJson(String stage, String logs) {
    if (logs == null) logs = "";
    logs = logs.replace("\\", "\\\\")
        .replace("\"", "\\\"")
        .replace("\n", "\\n")
        .replace("\r", "\\r");
    return String.format("{\"stage\":\"%s\",\"logs\":\"%s\"},\n", stage, logs);
}

public String getJsonReport() {
    if (!jsonReport.isEmpty()) {
        jsonReport.setLength(jsonReport.length() - 2);
    }
    return "[" + jsonReport + "]";
}
}

```

XMLReportVisitor

```

package com.example.demo.service.executor.visitor;

import com.example.demo.model.submission.SubmissionEntity;
import com.example.demo.service.executor.stage.BuildStageExecutor;
import com.example.demo.service.executor.stage.LinterStageExecutor;
import com.example.demo.service.executor.stage.TestStageExecutor;
import org.springframework.stereotype.Component;

@Component("XMLVisitor")
public class XMLReportVisitor implements Visitor {
    private StringBuilder xmlReport;

    @Override
    public void visitBuild(BuildStageExecutor build) {
        String logs = build.getLogs();
        xmlReport.append(LogsToXml(logs));
    }

    @Override
    public void visitTest(TestStageExecutor test) {
        String logs = test.getLogs();
        xmlReport.append(LogsToXml(logs));
    }

    @Override
    public void visitLinter(LinterStageExecutor linter) {
        String logs = linter.getLogs();
    }
}

```

```

        xmlReport.append(LogsToXml(logs));
    }

    @Override
    public String generateReport() {
        return xmlReport.toString();
    }

    private String LogsToXml(String logs) {
        if (logs == null) logs = "";
        logs = logs.replace("&", "&amp;")
            .replace("<", "&lt;")
            .replace(">", "&gt;")
            .replace("\\", "&quot;")
            .replace("'", "&apos;");
        return "<logs><![CDATA[" + logs + "]]></logs>\n";
    }
}

```

PipelineExecutor

```

package com.example.demo.service.executor;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class PipelineExecutor {

    private final SubmissionService submissionService;
    private final BuildStageExecutor buildStage;
    private final TestStageExecutor testStage;
    private final LinterStageExecutor linterStage;
    private final Visitor visitor;

    @Scheduled(fixedDelay = 5000)
    public void execute() {
        submissionService.getAllSubmitted().forEach(submitted -> {
            SubmissionEntity submission =
submissionService.findSubmissionById(submitted.getId());
            if(submission.getStatus() == SubmissionEntity.Status.SUBMITTED) {
                List.of(buildStage, testStage, linterStage).forEach(pipelineElement ->
{
                    pipelineElement.accept(visitor);
                    pipelineElement.execute(submission);
                });
            }
        });
    }

    public String generateReport(){
        return visitor.generateReport();
    }
}

```

```
}
```

Код всередині кожного компоненту (Build, Test, Linter)

```
@Override  
public void accept(Visitor visitor) {  
    visitor.visitBuild(this);  
}
```

```
@Override  
public void accept(Visitor visitor) {  
    visitor.visitTest(this);  
}
```

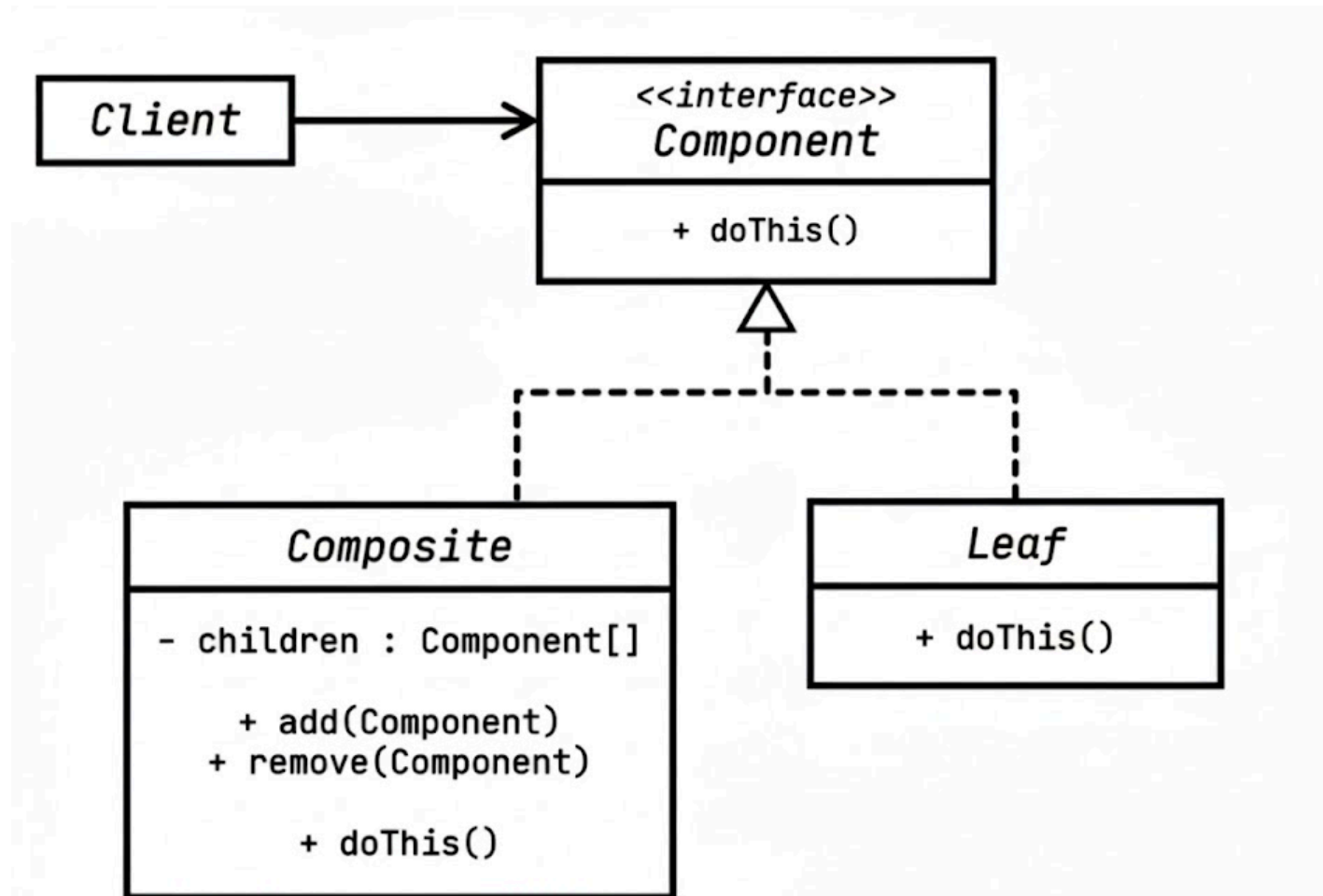
```
@Override  
public void accept(Visitor visitor) {  
    visitor.visitLinter(this);  
}
```

6. Питання до лабораторної роботи

1. Яке призначення шаблону «Композит»?

Шаблон «Посередник» призначений для централізації взаємодії між багатьма об'єктами системи через один спеціальний об'єкт — посередник. Завдяки цьому кожен компонент системи не потребує знати про всі інші об'єкти, а лише взаємодіє з посередником, який перенаправляє повідомлення, виклики або дані.

2. Нарисуйте структуру шаблону «Композит»



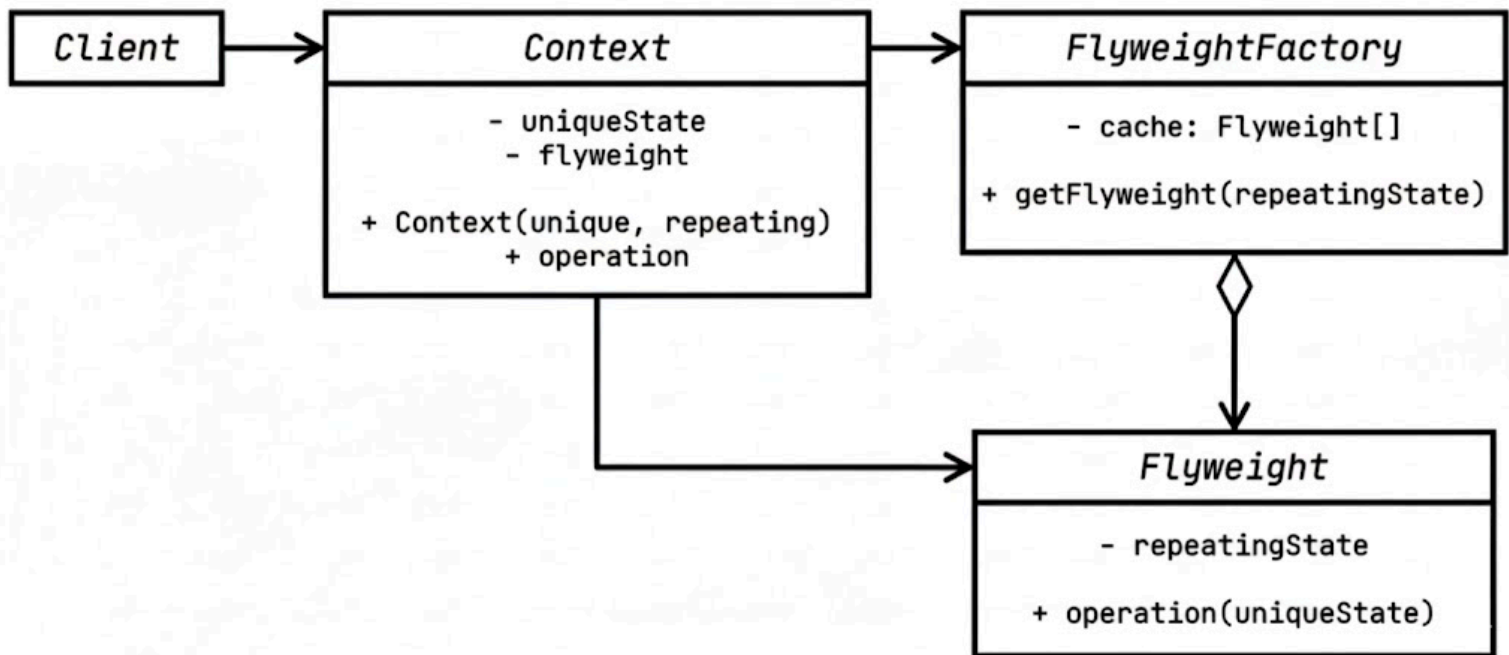
3. Які класи входять в шаблон «Композит», та яка між ними взаємодія?

У шаблон «Композит» входять три основні класи: **Component** — абстрактний клас або інтерфейс для всіх об'єктів, **Leaf** — кінцевий об'єкт дерева без підлеглих, і **Composite** — об'єкт, який може містити дочірні елементи. Клієнт працює через **Component**, не розрізняючи **Leaf** і **Composite**. **Composite** делегує операції своїм дітям, створюючи рекурсивну структуру.

4. Яке призначення шаблону «Легковаговик»?

Шаблон «Легковаговик» призначений для зменшення витрат пам'яті при створенні великої кількості схожих об'єктів шляхом спільного використання спільних внутрішніх даних, а відмінні дані зберігаються окремо для кожного об'єкта.

5. Нарисуйте структуру шаблону «Легковаговик».



6. Які класи входять в шаблон «Легковаговик», та яка між ними взаємодія?

У шаблон «Легковаговик» входять класи **Flyweight** — для спільних даних, **ConcreteFlyweight** — реалізація **Flyweight**, **UnsharedConcreteFlyweight** — для унікальних даних, і **FlyweightFactory** — для створення та повторного використання об'єктів. Клієнт отримує **Flyweight** через фабрику, яка забезпечує повторне використання, а **Flyweight** оперує спільними даними, отримуючи у клієнта зовнішні дані.

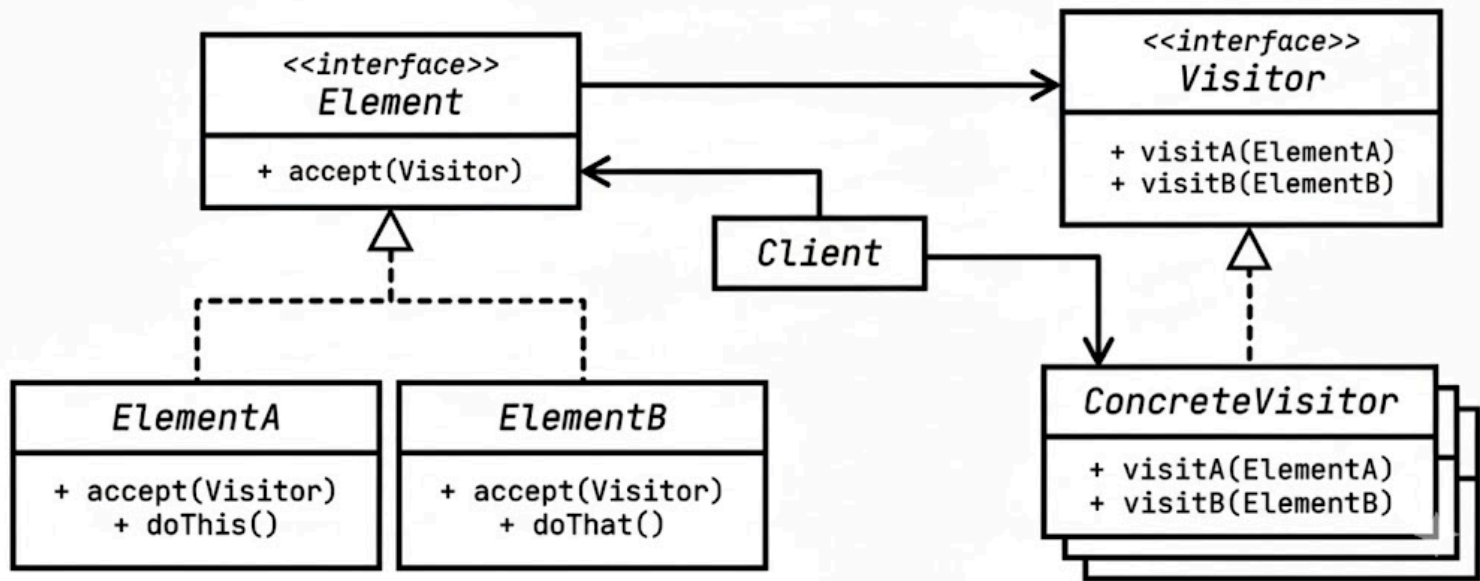
7. Яке призначення шаблону «Інтерпретатор»?

Шаблон «Інтерпретатор» призначений для визначення граматики мови та створення інтерпретатора, який аналізує та виконує вирази цієї мови. Він дозволяє представляти складні конструкції у вигляді об'єктної структури та легко додавати нові правила.

8. Яке призначення шаблону «Відвідувач»?

Шаблон «Відвідувач» призначений для відокремлення операцій від об'єктів, над якими ці операції виконуються. Він дозволяє додавати нові операції без зміни класів об'єктів, підтримуючи відкритість для розширення та спрощуючи роботу зі складними структурами даних.

9. Нарисуйте структуру шаблону «Відвідувач».



10. Які класи входять в шаблон «Відвідувач», та яка між ними взаємодія?

У шаблон «Відвідувач» входять класи **Visitor** — інтерфейс для операцій, **ConcreteVisitor** — реалізація логіки, **Element** — інтерфейс для об'єктів, **ConcreteElement** — реалізація елемента, **ObjectStructure** — колекція елементів. Клієнт передає **ConcreteVisitor** через метод `accept`, елемент викликає відповідну операцію `visitor`.

Висновок

У ході лабораторної роботи було реалізовано шаблон «Відвідувач» для генерації звітів по етапах пайплайну: Build, Test та Linter. Було створено інтерфейс Visitor з методами visitBuild, visitTest та visitLinter, а також два конкретні відвідувачі — JsonReportVisitor для формування звіту в JSON і XMLReportVisitor для формування звіту в XML. Клас PipelineExecutor координує виконання етапів, а кожен етап реалізує метод асерт, який викликає відповідний метод відвідувача. Це дозволяє формувати звіти без зміни коду етапів, легко додавати нові формати звітів, відокремлює логіку обробки від самих компонентів і спрощує підтримку та розширення системи.

Backend: <https://github.com/makszaranik/trpz-8>